ASSIGNMENT 3

Q3 - Transaction Validator

JavaScript solution

```
// q3.js
"use strict";

const transactions = [
  { id: 1, amount: 2000 },
  { id: 2, amount: -500 },
  { id: 3 },
  null
];

const valid = [];
const invalid = [];

for (let index = 0; index < transactions.length; index++) {
  const transaction = transactions[index];
  try {
    if (transaction === null) {
      const error = new Error("Null transaction entry");
      error.name = "NullEntryError";
      throw error;
    }
    if (transaction.id === undefined || transaction.amount === undefined) {
      const error = new Error("Missing id or amount");
      error.name = "MissingFieldError";
      throw error;
    }
    if (transaction.amount < 0) {
      const error = new Error("Amount cannot be negative");
      error.name = "NegativeAmountError";
      throw error;
    }
    valid.push(transaction);
    console.log(`Valid transaction: id=${transaction.id}, amount=${transaction.amount}`);
  } catch (error) {
    invalid.push({ index, transaction, type: error.name, message: error.message });
    console.log(`Invalid item at index ${index}: ${error.name} - ${error.message}`);
  }
}

console.log("Valid transactions:", valid);
console.log("Invalid transactions:", invalid);
console.log(`Summary: ${valid.length} successful, ${invalid.length} failed.`);

// Debugging: set a breakpoint on the first line inside the loop.
// Watch: index, transaction, valid, invalid, error.
// Step Over to inspect each record; Step Into is useful when calling helper functions.
// Call stack shows the current function and its caller.
```

Explanation / Debug notes

Custom error names let the catch block categorize failures. The null record is checked before reading properties, preventing a TypeError. For a debugger session, set a breakpoint inside the loop and watch index, transaction, valid, and invalid.

Run the code in the browser console or Node.js. Save each solution in its own .js file as required.